Curso de Desarrollo Web - Guia de Ejercicios Prácticos
(HTML - CSS).

Parte 1 Dominando CSS Grid Layout

CSS Grid es un sistema de maquetación bidimensional diseñado para manejar tanto filas como columnas simultáneamente. A diferencia de Flexbox (que alinea elementos en una sola dirección), Grid nos permite crear "tableros" o "rejillas" complejas de manera limpia y ordenada.

```css
.contenedor-grid {
    display: grid;
    grid-template-columns: 200px 200px 200px;
    grid-template-rows: 100px 100px;
    gap: 15px;
}
```

Nota : Para activar Grid, convertimos un contenedor en una cuadrícula y definimos sus pistas (columnas y filas)

Ejercicio N° - 1 : Creando tu Primera Cuadrícula con `grid-template-columns`

Imagina que estás organizando una galería de 4 fotos o productos en tu página web. En lugar de usar flotados o tablas antiguas, utilizaremos CSS Grid para ordenar los elementos en columnas automáticas. Vamos a aprender a usar la famosa unidad fraccional `(fr)` que reparte el espacio de forma inteligente.

Paso 1: Crea un archivo **index.html** con la siguiente estructura :

Paso 2 : Crea un archivo **styles.css.** Estos son los estilos fijos que mantendremos para que la estructura se vea ordenada :

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Práctica de CSS Grid - Columnas</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>

    <div class="grid-container">
        <div class="item">1</div>
        <div class="item">2</div>
        <div class="item">3</div>
        <div class="item">4</div>
    </div>

</body>
</html>
```

```css
body {
    font-family: Arial, sans-serif;
    background-color: #f8fafc;
    padding: 20px;
}

.grid-container {
    display: grid;
    background-color: #e2e8f0;
    border: 2px dashed #94a3b8;
    padding: 15px;
    gap: 15px;
    grid-template-columns: 1fr 1fr;
}

.item {
    background-color: #3b82f6;
    color: white;
    height: 100px;
    display: flex;
    justify-content: center;
    align-items: center;
    font-size: 24px;
    font-weight: bold;
    border-radius: 8px;
}
```

Paso 3 : Al visualizar el proyecto en el navegador obtendremos el resultado de la imagen. Elegir qué **opción** y propiedad es la adecuada dentro de las clases **.grid-container :**



Opción A: justify-content: flex-start;
Opción B: justify-content: space-between;
Opción C: grid-template-columns: 1fr 1fr 1fr 1fr;
Opción D: grid-template-columns: 1fr auto;

Paso 4 : Resultados esperados. Selecciona qué Opción es la adecuada para que permita crear tres columnas donde la del centro es el doble de ancha que las otras dos.



Opción A: grid-template-columns: 1fr 1fr 1fr 1fr;
Opción B: grid-template-columns: 1fr auto;
Opción C: gred-template-columns: 1fr 2fr 1fr;

Ejercicio N° - 2 : Filas y Alturas con `grid-template-rows` y `gap`

Una vez que sabemos organizar columnas, es hora de controlar el espacio vertical. Vamos a estructurar un diseño tipo "Panel de Control" o tablero, definiendo alturas específicas para las filas y separando los elementos de forma limpia con la propiedad `gap`.

Paso 1 : Crear un archivo **index.html** con la siguiente estructura :

Paso 2 : Crear un archivo **styles.css** con los siguientes con las siguientes clases y propiedades :

```css
.grid-panel {
    display: grid;
    background-color: #f1f5f9;
    border: 2px solid #cbd5e1;
    padding: 15px;
    grid-template-columns: 1fr 1fr;
    grid-template-rows: 80px 150px;
    gap: 20px;
}

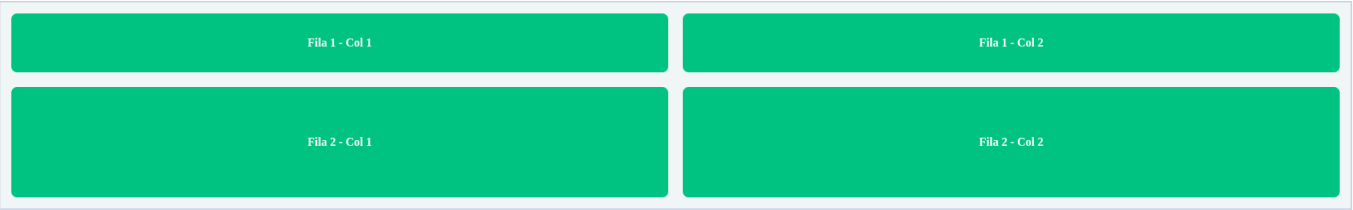
.caja {
    background-color: #10b981;
    color: white;
    display: flex;
    justify-content: center;
    align-items: center;
    font-weight: bold;
    border-radius: 8px;
}
```

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Práctica de CSS Grid - Columnas</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>

    <div class="grid-panel">
        <div class="caja">Fila 1 - Col 1</div>
        <div class="caja">Fila 1 - Col 2</div>
        <div class="caja">Fila 2 - Col 1</div>
        <div class="caja">Fila 2 - Col 2</div>
    </div>

</body>
</html>
```

Paso 3 : Al visualizar el proyecto en el navegador obtendremos el resultado de la imagen. Verás una cuadrícula de 2x2 donde la primera fila es bajita (80px) y la segunda fila es notablemente más alta (150px), separadas por un espacio limpio gracias al `gap`.



Ejercicio N° - 3 : Estirando Elementos con `grid-column` (El superpoder de Grid)

En Grid, los elementos no tienen por qué medir solo 1 celda. Podemos hacer que un elemento "se estire" y abarque varias columnas o filas a la vez. Esto es genial para crear banners superiores o tarjetas destacadas.

Paso 1 : Crear un archivo **index.html** con la siguiente estructura en la cual se muestra en la imagen:

Paso 2 : Crear un archivo **styles.css** con las siguientes clases y propiedades en la cual se pueden observar en la imagen:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="./styles3.css">
    <title>Practica CSS Grid</title>
</head>
<body>

    <div class="grid-hero">
        <div class="item banner">Banner Principal (Ocupa todo el ancho)</div>
        <div class="item">Tarjeta 1</div>
        <div class="item">Tarjeta 2</div>
    </div>

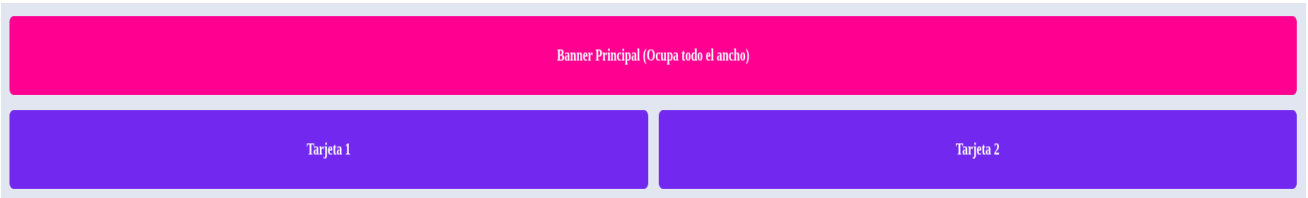
</body>
</html>
```

```css
.grid-hero {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
    background-color: #e2e8f0;
    padding: 15px;
    border-radius: 8px;
}

.item {
    background-color: #6366f1;
    color: white;
    padding: 30px;
    text-align: center;
    font-weight: bold;
    border-radius: 6px;
}

/* El elemento destacado */
.banner {
    background-color: #ec4899;
    grid-column: 1 / 3;
}
```

Paso 3 : Al visualizar el proyecto en el navegador obtendremos el resultado de la imagen. Arriba, el banner rosado cruza horizontalmente ocupando el ancho completo de las dos columnas. Abajo, las Tarjetas 1 y 2 se colocan ordenadamente una al lado de la otra. Elige qué opción es la adecuada dentro de la clase **.grid-hero** en la cual permita tener 3 columnas:



Opción A: position: sticky;
Opción B: position: absolute;
Opción C: grid-column: 2/5;
Opción D: grid-column: 1/5;
Opción E: grid-template-columns: 1fr 1fr 1fr;
Opción F: grid-template-columns: 1fr 1fr auto;
Opción G: flex-direction: row-reverse;

Paso 4 : Visualizar la imagen de los resultados esperados. Analizar que opción es la correcta para obtener los resultados esperados :

Opción A: Cambiar las propiedades en la clase grid-hero
Opción B: Crear un nuevo archivo HTML con una estructura diferente
Opción C: Cambiar los colores de las cajas
Opción D: Utilizar la propiedad gap para aumentar el espaciado de las cajas en su clase .item
Opción E: Reducir el tamaño proporcional de las columnas dentro de la clase .banner
Opción F: Anexar 2 div adicionales a la estructura de el HTML
Opción G: No se la respuesta, pero no hay problema le pregunto a mi tutor



Ejercicio N° - 4 : Diseña una Plantilla Web Clásica (Header, Sidebar, Main, Footer)

¡Es tu turno de brillar con CSS Grid!

Imagina que estás maquetando la estructura clásica de un blog o sitio web profesional. Tu objetivo es crear un contenedor Grid que organice 4 secciones principales exactamente como se ve en la estructura visual de abajo:

Req 1 - Header (Cabecera): Debe ocupar todo el ancho superior de la pantalla.

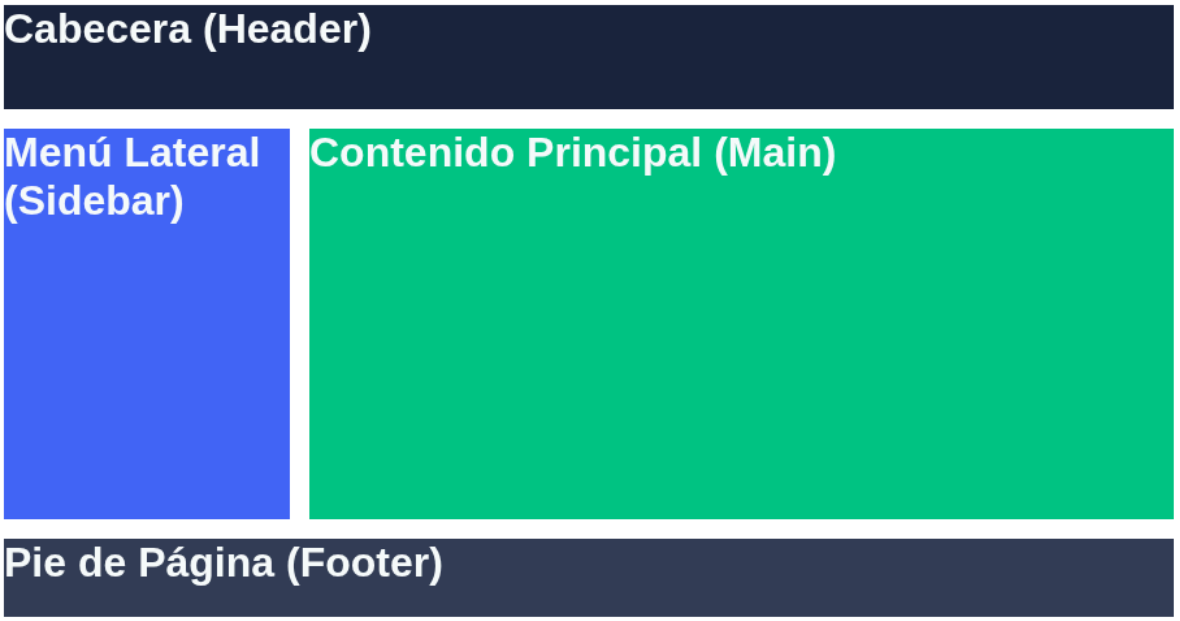
Req 2 - Sidebar (Menú lateral izquierdo): Una barra vertical angosta a la izquierda.

Req 3 - Main (Contenido principal): La sección central grande donde van los artículos.

Req 4 - Footer (Pie de página): Debe ocupar todo el ancho inferior de la pantalla.

Pista 1 **:** Define un contenedor con dos columnas (una estrecha para el menú y otra ancha para el contenido: `grid-template-columns: 250px 1fr`).

Pista 2 : Usa la propiedad `grid-column: 1 / 3;` tanto en el `header` como en el `footer` para forzarlos a estirarse cubriendo ambas columnas de lado a lado.



" Aprende Creando, Crea Aprendiendo "